

Sistemas Operacionais



Educação a Distância
Cruzeiro do Sul Educacional
Campus Virtual

Material Teórico



Gerenciamento de Processos

Responsável pelo Conteúdo:

Prof. Ms. Claudney Sanches Junior

Revisão Textual:

Profa. Dra. Patrícia Silvestre Leite Di Iório

UNIDADE

Gerenciamento de Processos



- Gerenciador de Processos
- Parâmetros de Escalonamento
- Interrupção
- Threads



**Objetivo de
APRENDIZADO**

O objetivo desta unidade será estudar os conceitos de processos dos SO. A unidade apresenta a definição de processos e mostra o gerenciamento dos mesmo.

Detalha os estados do processos e apresenta o conceito de threads. Neste contexto, espera-se que ao final da unidade você seja capaz de entender o gerenciamento de processos e interrupção.

Para que possa entender os conceitos de processos e escalonamento do SO, esta unidade está organizada da seguinte forma:

- a seção 2, apresenta o gerenciador de processos;
- a seção 3, mostra os parametros de escalonamento;
- a seção 4, detalha a interrupção;
- a seção 5, apresenta as threads.

Ao final do estudo e das atividades desta unidade, você deve ser capaz de:

- entender e caracterizar processos em SOs;
- conhecer o escalonador e seus principais algoritmos.

Contextualização



Gerenciador de Processos



Para resolver as dificuldades de multiprogramação e tempo compartilhado ou *multiprogramming* e *time-sharing*, há cinco grandes assuntos que todo SO moderno deve considerar. Vamos abordar o primeiro que é o gerenciamento de processos. Os SOs multiprogramados devem proporcionar o melhor aproveitamento da CPU quer ela tenha um processador ou vários processadores dividindo o tempo da mesma, para que vários programas rodem em fila, com intervalos de tempo tão pequenos e com trocas tão rápidas que acabam dando a ilusão ao usuário que todos estão sendo executados simultaneamente, independentemente se estão trabalhando fisicamente em paralelo ou logicamente em paralelo através do SO.

Um programa de computador é o algoritmo escrito em uma linguagem de programação, como a linguagem C ou linguagem Java, com o objetivo de resolver um determinado problema. Quando um programa está em execução em uma CPU receberá o nome de processo. Processo é um termo mais genérico do que *job* ou *task*, introduzido para obter uma maneira sistemática de monitorar e controlar a execução de um programa. O conceito de processo é dinâmico em contraposição ao conceito de programa que é estático. O programa reside em no disco, não faz nenhuma ação sem entrar em execução enquanto que o processo reside na memória principal da máquina e está no processo de execução. O processo consiste em um programa executável associado aos seus dados e ao seu contexto de execução. Nem sempre um programa irá equivaler a apenas um processo, pois existe SO que permitem a reentrância, permitindo a um programa gerar diversos processos. Uma característica marcante do processo é que o programa ou código que o gera não pode apresentar nenhuma característica de suposição de temporização. Em geral, o SO determina a fatia de tempo entre os processos e esta fatia de tempo é imprevisível. Um algoritmo de escalonamento determina quando um processo irá parar e outro irá entrar em execução.

O contexto de execução de um processo é o conjunto de dados necessários à sua execução, como por exemplo, a identificação do processo chamado de *pid*, todos os conteúdos dos registradores dos processadores tais como o contador de programa ou simplesmente *PC* - *Program Counter*, os ponteiros *SP* - *Stack Pointer*, as variáveis e dados armazenados na memória, a lista de arquivos que estão sendo utilizados, tempo de CPU disponível, prioridade de execução, eventos que o processo pode estar esperando entre outros. Essas informações são fundamentais para que um processo interrompido pelo escalonador possa voltar a executar exatamente a partir do ponto de parada sem perda de dados ou inconsistências. Tais informações são armazenadas em estruturas de dados conhecidas como tabela de processos ou descritor de processos ou bloco descritor de programa. O bloco descritor de programa (BCP) consiste de uma estrutura de dados contendo informações importantes sobre o processo. A figura 1 apresenta o BCP contendo os dados do contexto de execução.

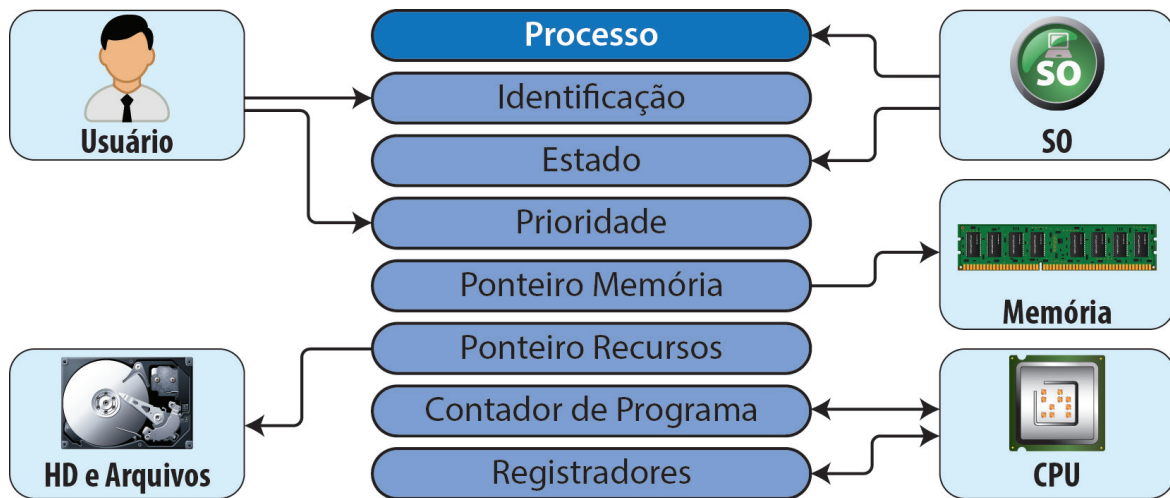


Figura 1. BCP contendo dados do contexto de execução

A troca entre processos concorrentes pela disputa da CPU chama-se *mudança de contexto*. Quando o processo é interrompido pelo sistema operacional, seu contexto é salvo no seu BCP. Ao retornar a execução, o sistema operacional restaura o contexto do processo, o qual continua a executar como se nada tivesse ocorrido. A figura 2 apresenta os sistemas logicamente paralelos concorrentes em uma única CPU.

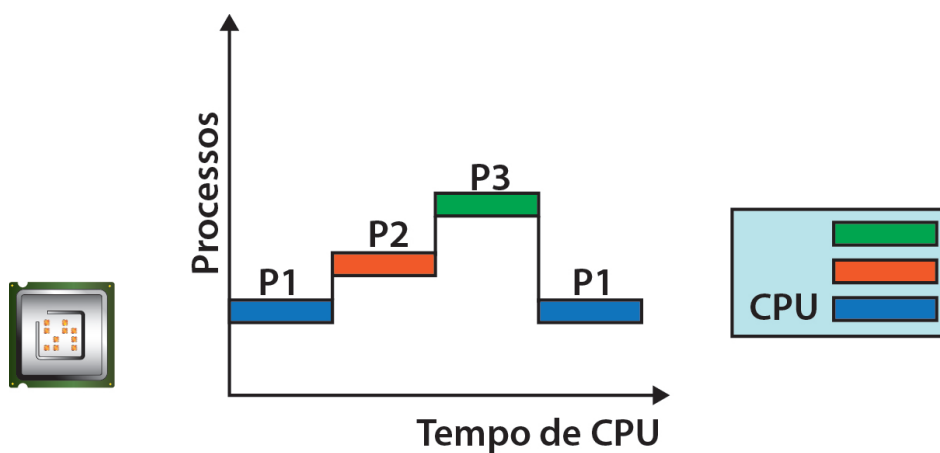


Figura 2. Sistemas Lógicamente Paralelos

Um dos principais requisitos de um SO é intercalar ou fazer a mudança de contexto entre processos visando maximizar a utilização da CPU fornecendo um razoável tempo de resposta. Atualmente, tem se popularizado os computadores com múltiplas CPUs que compartilham os demais recursos da máquina, tais como memória principal e dispositivos de entrada e saída. A figura 3 apresenta os processos em um sistema fisicamente paralelo.

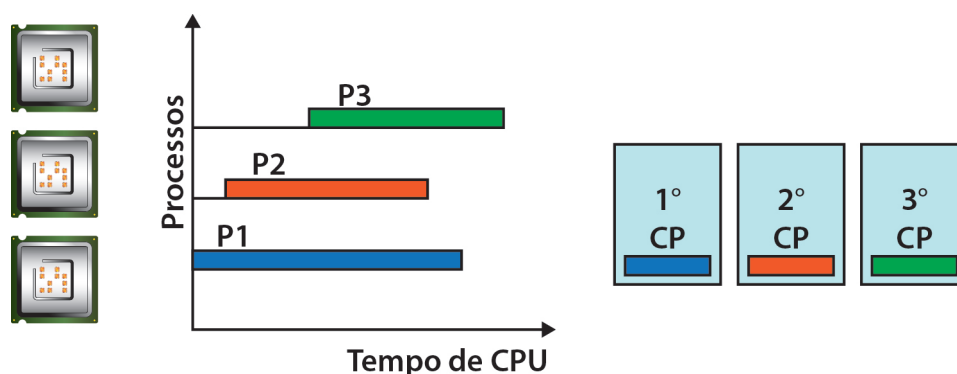


Figura 3. Sistemas Fisicamente Paralelos

O SO deve evitar que os processos entrem em *deadlock* e permitir a criação e comunicação entre processos. Os processos após sua criação podem assumir alguns estados. Os primeiros SO multiprogramados tinham a previsão de poucos estados, como o criação, o em execução, o estado pronto, o estado bloqueado e o estado encerrado, conforme ilustrado na figura 4.

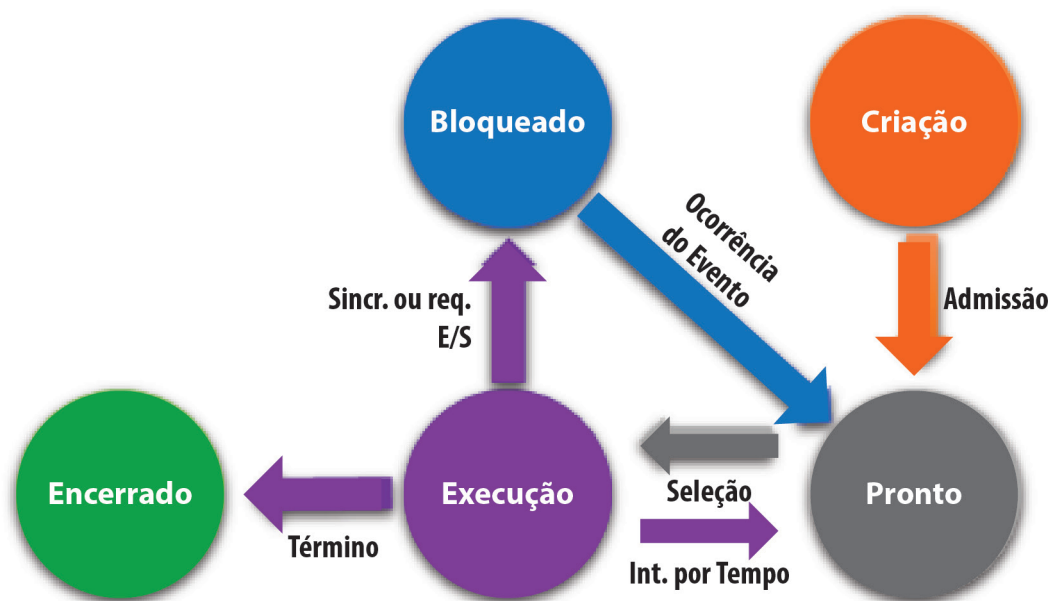


Figura 4. Estados dos processos de um SO

Quando um usuário solicita a execução de um programa o SO cria um processo com a identificação do programa e deverá criar o contexto de execução para poder escalonar ou seja colocar ele em execução. Quanto o SO conclui todos os preparativos para rodar o processo ele muda o estado do processo de criação ou indefinido para pronto que este fica a disposição do escalonador do *kernel* do SO.

O software escalonador deverá cuidar dos processos definindo quem deverá permanecer em execução. Uma das políticas para o escalonador é dar fatia de tempo iguais a todos os processos mudando o estado do processo de pronto para em execução. Quando o processo está em execução, ele tem todos os recursos da CPU para utilizar, ou seja, está em seu estado progressivo tendo um andamento normal. Enquanto não ocorrer uma chamada de entrada e saída ou enquanto não findar sua fatia de tempo o processo deverá permanecer neste estado.

Ao terminar sua quota de tempo para execução, o escalonador o interrompe, colocando-o no estado pronto, ou seja, o processo só não está em execução pelo fato da CPU estar sendo utilizada por outro processo. Ao chegar novamente sua vez de executar, o escalonador invoca ou acorda o processo permitindo sua continuidade. Um processo pode ser bloqueado se os dados de entrada ou saída, ainda, não estiverem disponíveis ou ele estiver parado à espera da ocorrência de um evento ou ele não poder continuar sua execução ou andamento progressivo.

O processo pode ir para o estado bloqueado informando ao escalonador que entrou em espera, simplesmente, enviando um sinal solicitando que outro processo entre em execução ou por decisão do escalonador. O processo é desbloqueado por um evento externo como, por exemplo, a chegada dos dados ou sinalização de outro processo e ele então passa para o estado pronto.

Para efetuar o compartilhamento da CPU entre processos, o SO possui duas filas de controle: a de processos prontos ou *ready-list* e a de processos bloqueados ou *blocked-list*. A manipulação dessas filas depende da política de escalonamento adotada pelo sistema. Um escalonador poderia funcionar da seguinte forma: quando a CPU torna-se disponível, o primeiro elemento da fila de processos prontos é retirado e inicia sua execução. Caso seja bloqueado, este irá para o final da fila de processos bloqueados. Se a sua quota de execução se esgotar, este será retirado da CPU e colocado no final da lista de processos prontos.

O escalonador ou *scheduler* do SO é o elemento responsável pela alocação de processo(s) no(s) processador(es), definindo sua ordem de execução. A política de escalonamento ou *scheduling* em inglês é um problema complexo e depende do tipo de sistema suportado e da natureza das aplicações. Em sistemas do tipo *batch*, o escalonamento era feito simplesmente selecionando o próximo processo na fila de espera. Em sistemas multiusuário de tempo repartido, geralmente combinados a sistemas em *batch*, o algoritmo de escalonamento deve ser mais complexo em virtude da existência de diversos usuários solicitando serviços e da execução de tarefas em segundo plano ou *background*.

Parâmetros de Escalonamento



Em um sistema multitarefa, vários programas compartilham a mesma CPU e, portanto, num dado instante de tempo somente um processo estará executando e vários processos podem estar a espera de sua fatia de tempo. Isto introduz filas de processos no estado de pronto. Quando estoura a fatia de tempo do processo em execução, o SO deve decidir qual processo da fila de prontos deverá receber a CPU. A parte do SO responsável por decidir qual processo, dentre os prontos, deve ganhar o direito de uso da CPU é denominada escalonador de processos ou *scheduler*. O escalonador ordena a fila de prontos de acordo

com sua política de escalonamento, a qual pode levar em consideração a prioridade do processo, a sua ordem de chegada no sistema, seu prazo para ser atendido etc.

O escalonamento de um SO pode ser classificado como preemptivo e não-preemptivo. O escalonamento é dito preemptivo se o processo em execução na CPU puder ser interrompido para a execução de outro processo. Preempção é utilizada em sistemas multitarefa para garantir que todos os processos possam progredir e para evitar que um processo monopolize a CPU. E o escalonamento é dito não-preemptivo se durante a execução de um processo a CPU não puder ser liberada para outro processo.

Diversos critérios devem ser considerados para a implementação de um bom algoritmo de escalonamento. Alguns deles são:

- Justiça: garantir que cada processo tenha direito de acesso a CPU;
- Eficiência: procurar maximizar a utilização da CPU;
- Tempo de Resposta: procurar minimizar o tempo de resposta para aplicações interativas. O Tempo de resposta é o tempo decorrido entre o momento em que um usuário submete uma tarefa ao sistema e instante em que ele recebe de volta os resultados;
- *Throughput* ou vazão: maximizar o número de tarefas processadas por unidade de tempo;
- *Turnaround* ou tempo de utilização da CPU: a gestão estratégica do tempo de utilização da CPU por *job* procura minimizar o tempo de execução das tarefas do tipo lote.

Um dos algoritmos de escalonamento mais conhecidos é o **FIFO** do inglês *First In First Out* ou em português primeiro a entrar na fila é o primeiro a sair. Em um SO do tipo FIFO os processos vão sendo colocados na fila e retirados por ordem de chegada. A ideia fundamental é a de uma fila, em que só se pode inserir um novo elemento no final da fila e só se pode retirar o elemento do início. Neste algoritmo, os processos são selecionados a partir da sua ordem de chegada, ou seja, o primeiro a chegar é o primeiro a ser servido, daí o motivo do mesmo algoritmo receber o nome de *First Come First Served (FCFS)*. O mecanismo adotado pelo escalonador é não-preemptivo, ou seja, as tarefas ao conseguirem a CPU executam até o final. Processos maiores fazem com que processos menores esperem em demasia e devido a igualdade total entre tarefas, processos mais importantes não tem acesso privilegiado a CPU. A falta de garantia quanto ao tempo de resposta torna-o inadequado para sistemas interativos, porém pode ser utilizado em sistemas *Batch*.

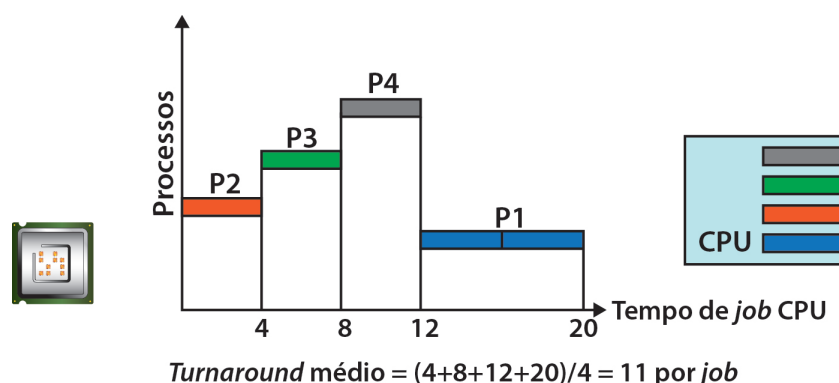
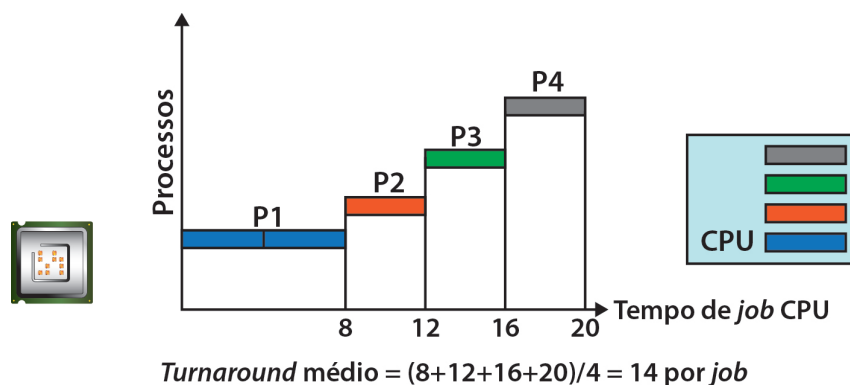
Outro algoritmo muito conhecido para o escalonamento é *Round Robin* ou circular. É um algoritmo antigo, porém justo e simples. O processo é colocado para executar na ordem em que fica pronto para execução, porém só pode executar por uma fatia de tempo, após a qual o processo é interrompido e, caso ainda não tenha terminado sua execução, volta para o final da fila de prontos. Um dos problemas do algoritmo *Round Robin* é determinar a fatia de tempo ou como alguns autores preferem chamar a fatia de tempo de quantum. Um quantum muito pequeno pode levar a sucessivas trocas de contexto reduzindo a eficiência do processador. Um quantum muito grande pode se tornar inviável para sistemas interativos elevando o tempo de resposta.

Como nem sempre todos os processos tem a mesma prioridade o algoritmo de escalonamento com prioridade passa a ser interessante. A ideia do algoritmo é simples, a cada processo é associada uma prioridade e o processo pronto com maior prioridade é executado primeiro. Para evitar o monopólio do processador pelo processo com maior prioridade a cada quantum o sistema deverá decrementar

a prioridade do processo em execução. O escalonamento por prioridade pode ser por prioridade estáticas ou por prioridade dinâmicas. A maioria dos SO de tempo compartilhado implementa um esquema de prioridades. Pode ser conveniente, algumas vezes, agrupar os processos em classes de prioridades e usar o escalonamento entre as classes e o round robin dentro das classes.

Um dos mais antigos escalonadores com prioridade foi projetado para o SO CTSS. O CTSS só mantinha na memória principal um processo pequeno, logo foi necessário atribuir um quantum longo para processos que utilizavam muito a CPU como uma forma de reduzir a *swap*. Para evitar o tempo de resposta ruim o SO dividiu os processos em classes de prioridades. As classes com maior prioridade rodavam 1 quantum, a próxima 2 quantum, a outra 4 quantum e assim sucessivamente. Outro SO que atribuiu classes de prioridade foi o XDS 940, onde as classes terminal e de entrada e saída tinha maior prioridade que as demais.

O algoritmo mais apropriado para sistemas que executam jobs em *batch* é o de menor tarefa primeiro ou *Shortest Job First* (SJF). Neste algoritmo, o processo com menor tempo de execução previsto é o próximo escolhido para executar. O algoritmo é não-preemptivo e visa reduzir o tempo médio de espera das tarefas. O maior problema do algoritmo reside na estimativa prévia do tempo de execução do processo. Alguns processos *em batch* que executam regularmente como folha de pagamento e contabilidade, tem geralmente tempos de execução conhecidos, porém processos interativos normalmente não usufruem dessa propriedade. Considere o caso ilustrado na figura 5 que apresenta 4 jobs com seus tempos de execução e compare com o a figura 6 que apresenta os mesmos jobs mas agora com o algoritmo da menor tarefa primeiro e observe o tempo média menor de CPU por job.



É interessante notar que o algoritmo do menor *job* só conduz resultados ótimos se todos os *jobs* estão disponíveis ao mesmo tempo. Essa abordagem pode ser utilizada em sistemas interativos, tendo que considerar que cada comando enviado pelo usuário passa a ser um *job* e deverá ter o seu tempo estimado a partir de seu comportamento no passado. Uma maneira de estimar com mais facilidade é utilizar a técnica chamada de *aging*. Suponha que o tempo para um comando de terminal na primeira vez que foi chamado seja T_0 . Agora, na rodada seguinte o tempo medido seja T_1 . Podemos atualizar nossa estimativa considerando $aT_0 + (1-a)T_1$. A escolha fácil de implementar é fazer $a = \frac{1}{2}$. Assim já na quarta estimativa teríamos:

1º - T_0

2º - $T_0/2 + T_1/2$

3º - $T_0/4 + T_1/4 + T_2/2$

4º - $T_0/8 + T_1/8 + T_2/4 + T_3/2$

Note que o peso de T_0 ou o tempo inicial com provavelmente o maior erro de estimativa caiu a $1/8$ e a estimativa tende a ficar mais exata a cada interação.

Para sistemas de tempo real é comum encontrar o escalonamento por prazo ou *deadline*. Os processos são escolhidos para execução em função de suas urgências. Esta política é utilizada em sistemas *real-time* onde, a partir do momento em que o processo entra na fila de prontos, inicia seu prazo para receber a CPU. Caso não seja atendido dentro desse prazo pode haver perda de informação. Os processos com escalonamento garantidos são similares aos escalonadores de tempo real, pois é feita uma promessa ao usuário a respeito do desempenho que tem de ser cumprida. Muitos SO prometem que o escalonador irá atribuir a n usuários ativos $1/n$ da capacidade do processador.

Algumas vezes é importante separar o mecanismo de escalonamento da política de escalonamento. Por exemplo, quando um processo-pai possui vários processos-filhos, pode ser importante deixar o processo-pai interferir no escalonamento do processo filho. Para isso, deve-se ter um escalonador parametrizado, cujos parâmetros são passados pelos processos do usuário.

Se não existir memória disponível no sistema alguns processos devem ser mantidos em disco. Esta situação tem grande impacto no escalonamento devido ao atraso provocado pelo *swapping*. Uma forma prática para lidar com o *swapping* é através do escalonamento em dois níveis: um para o subconjunto dos processos mantidos na memória principal e outro para os processos mantidos em disco. O escalonador de mais baixo nível se preocupa em escolher qual processo da memória usará a CPU enquanto que o escalonador de mais alto nível se preocupa em fazer a troca dos processos da memória com os em disco.

Interrupção



Para que se possa realizar o escalonamento de processos, é necessário um mecanismo para interromper a execução do processo atualmente usando a CPU e fazer com que a CPU passe a executar as instruções do SO para que ele possa escalonar um novo processo. Este mecanismo é denominado interrupção.

Portanto, interrupção é um evento gerado pelo *hardware* ou por *software* e que altera a sequência na qual o processador executa as instruções. Por exemplo, toda máquina possui em clock que é um dispositivo de hardware que gera uma interrupção em tempos regulares no PC 18 vezes por segundo, fazendo com que a CPU pare de fazer o que estava fazendo e execute uma rotina que incrementa a hora e data do sistema. Outros tipos de interrupção são:

- interrupção de software: quando um processo solicita um serviço do SO;
- interrupção de entrada e saída (E/S): geradas por dispositivos de E/S tais como impressora, teclado, drive de disco, para indicar ao SO alterações no seu estado final de escrita ou leitura de dados, falha no dispositivo, etc;
- traps: interrupções causadas por erros na execução de programas, tais como tentativa de divisão por zero, *overflow*, entre outras.

O tratamento de interrupções é feito pelo SO. Toda vez que uma interrupção acontece, o SO assume o controle, salva o contexto de execução do processo interrompido no seu BCP, analisa a interrupção e passa o controle para a rotina de tratamento apropriada.

Threads



A ideia da *threads* é levar para dentro das aplicações a multitarefa. A ideia é permitir que determinados trechos de código do mesmo programa possam ser executados de forma concorrente ou paralelas. No modelo tradicional, um programa é composto de uma única *thread*, ou seja, existe apenas uma linha de execução e as instruções são executadas sequencialmente, do início ao final, uma após a outra. O conceito de *thread* implica a possibilidade de passar a executar vários pedaços de um processo ao mesmo tempo. Tudo se passa como se estivesse invocando a execução de uma função. No entanto, enquanto no modelo tradicional ao chamar a função se transfere o controle de fluxo de execução para ela, quando se utiliza uma *thread* simplesmente se ativa a execução da função e prossegue a execução da função invocadora. As duas *threads*, a função chamadora e a chamada, vão ser executadas ao mesmo tempo, concorrendo pelo uso dos recursos alocados ao processo.

Material Complementar

Com as Máquinas Virtuais (VM) que você instalou na Unidade I, inicie no SO Linux a gerência de seus processos. O SO Linux apresenta um dos melhores conjuntos de algoritmos para gerenciamento de processos. Pesquise os comandos, teste e experimente gerenciar seus processos no Linux.



Gerenciamento de Processos no Linux

- http://marcelotoledo.com/stuff/artigos/processos_no_linux/gerenciamento_de_processos_no_linux.html

Comandos para criar, gerenciar, monitor e eliminar processos

- <http://linuxelpi.blogspot.com.br/2010/06/gerenciamento-de-processos.html>

Referências

Tanenbaum, A.S. **Sistemas Operacionais Modernos**. 3º Edição. São Paulo. Pearson Prentice Hall, 2009.

Deitel, H.M. **Sistemas Operacionais**. 3º Edição. São Paulo. Pearson Prentice Hall, 2005.

[illegible]



Educação a Distância

Cruzeiro do Sul Educacional

Campus Virtual

www.cruzeirodosulvirtual.com.br

Campus Liberdade

Rua Galvão Bueno, 868

CEP 01506-000

São Paulo SP Brasil

Tel: (55 11) 3385-3000

